

Chapter 1

Introduction

In modern organizations, especially in defence, government, and critical infrastructure sectors, sensitive networks are often isolated from the Internet to protect confidential information from cyber threats. However, there is still a need to transfer files, documents, and software updates from Internet-connected systems to these secure Local Area Networks (LANs). This process introduces security risks such as malware infiltration, unauthorized access, and data tampering.

The Offline Secure File Transfer System is designed to enable safe and controlled transfer of files between an Internet-connected network and an isolated LAN environment. The system employs security mechanisms such as malware scanning, encryption, digital signatures, integrity verification, and access control to ensure that only authenticated and secure files are transferred. By using offline transfer methods and one-way communication mechanisms, the system minimizes the risk of cyberattacks while maintaining data confidentiality, integrity, and availability.

This project provides a reliable and secure solution for transferring sensitive information into air-gapped or high-security networks, making it suitable for defence, military, research, and government applications.

Key Features

1. Secure file transfer between Internet and air-gapped LAN networks.
2. AES-256 encryption to protect sensitive data during transfer.
3. SHA-256 hashing for file integrity verification.
4. Digital signatures to authenticate file sources.
5. Malware and virus scanning before file acceptance.
6. Role-Based Access Control (RBAC) for authorized user access.
7. Comprehensive audit logs for monitoring and accountability.
8. One-way data transfer mechanism to prevent reverse communication attacks.
9. File quarantine zone for additional security checks.
10. Automated alert system for suspicious or unauthorized activities.

Chapter 2

DATA DIODE :-

A data diode is a hardware security device that allows data to flow in only one direction between two networks or systems.

Think of it like a one-way valve for information:

Network A —————▶ Network B

Data can move from A to B, but it is physically impossible for data to travel back from B to A.

Why use a data diode?

Data diodes are used to protect highly secure networks from cyberattacks. For example:

1. Military and defense systems
2. Critical infrastructure (power plants, water treatment facilities)
3. Industrial control systems (SCADA)
4. Financial institutions
5. Government networks

How does it work ?

Unlike a firewall, which uses software rules to allow or block traffic, a data diode enforces one-way communication through hardware.

A common implementation uses:

1. A transmitter on one side
2. A receiver on the other side
3. No physical return path

Because the reverse path doesn't exist, malware or attackers on the receiving network cannot send commands back into the protected network.

Advantages:-

1. Extremely strong protection against remote attacks
2. Eliminates many classes of network intrusion
3. Helps meet high-security compliance requirements

Limitations:-

1. No direct two-way communication
2. Some protocols must be modified or proxied to work over a one-way link
3. Can be expensive and complex to deploy

Chapter 3

PROJECT AIM:-

The aim of this project is to design and implement an Offline Secure File Transfer System that facilitates the secure movement of files from an Internet-connected environment to a highly secured Local Area Network (LAN) without establishing any direct network connection between the two domains. The system is intended for use in security-sensitive environments such as defense organizations, government agencies, research laboratories, and critical infrastructure facilities where maintaining network isolation is essential for protecting confidential information.

The project aims to provide a reliable and controlled mechanism for file transfer by incorporating multiple layers of security, including malware scanning, file validation, encryption, integrity verification, digital signatures, user authentication, and access control. The system will ensure that only authorized and verified files are transferred into the secure LAN, thereby minimizing the risk of cyberattacks, malware infections, data tampering, and unauthorized access.

Additionally, the project seeks to maintain complete audit trails and logging of all transfer activities to support accountability, traceability, and compliance with organizational security policies. By using offline transfer methods such as secure removable media or one-way transfer technologies like data diodes, the system will preserve the air-gap between networks while enabling efficient and secure data exchange.

Ultimately, the aim of this project is to enhance the confidentiality, integrity, and availability of sensitive information by providing a robust, secure, and practical solution for transferring files between Internet-connected systems and isolated LAN environments.

Chapter 4

THREE ZONE MODEL:-

A Three-Zone Model divides the file transfer process into three separate security domains to ensure safe movement of data from the Internet to a secure LAN.

Zone 1: Internet Zone (Untrusted Network)

Purpose

1. Connects to external networks and the Internet.
2. Receives files from external users and sources.

Functions

1. File upload and download.
2. Initial file validation.
3. User authentication.
4. File submission to the intermediate zone.

Risks

1. Malware attacks.
2. Unauthorized access.
3. Data tampering.

Zone 2: Intermediate Zone (DMZ / Transfer Zone)

Purpose; Acts as a security buffer between the Internet Zone and the Secure Lan.

Functions

1. Antivirus and malware scanning.
2. File type validation.
3. SHA-256 hash generation and verification.
4. AES-256 file encryption.
5. Digital signature verification.
6. Audit logging and monitoring.
7. File quarantine for suspicious files.

Benefits

1. Prevents infected files from reaching the LAN.
2. Provides an additional security layer.
3. Maintains controlled data flow.

Zone 3: Secure LAN Zone (Trusted Network)

Purpose

Contains sensitive systems, confidential documents, and internal users.

Functions

1. Receives only verified files.
2. Performs final integrity checks.
3. Stores files securely.
4. Provides role-based access control.
5. Maintains transfer logs and records.
6. Security Requirements
7. No direct Internet access.
8. Strict authentication and authorization.
9. Continuous monitoring and auditing

Advantages of Three-Zone Model

1. Strong network isolation.
2. Reduced malware infection risk.
3. Better monitoring and auditing.
4. Controlled file transfer process.
5. Suitable for defense, military, government, and critical infrastructure environments.
6. Supports compliance with cybersecurity policies and standards.

For Your Project:-

Zone 1: Internet PC/Server

Zone 2: Transfer Gateway Server (Scanning + Encryption + Logging)

Zone 3: Secure LAN Server.

Chapter 5

6-STEPPPIPELINE MODEL:-

❖ File Submission:

- User uploads the file from the Internet zone.
- Metadata such as file name, size, and source are recorded.
- Initial file validation is performed.

❖ Security Scanning:

- Antivirus and malware scanning.
- File type verification.
- Detection of malicious scripts or hidden content.

❖ Quarantine & Validation:

- Suspicious files are isolated in a quarantine area.
- Security policies and access permissions are checked.
- Approved files move to the next stage.

❖ Data Diode Transfer:

- Files are transferred through a Data Diode (one-way communication device).
- Ensures data flows only from the lower-security zone to the higher-security zone.
- Prevents reverse communication and cyber attacks.

❖ Intermediate Zone Verification:

- Re-scan files after transfer.
- Verify file integrity using checksums/hashes (e.g., SHA-256).
- Log all transfer activities for auditing.

❖ Delivery to LAN:

- Approved files are released to the secure LAN.
- Authorized users access the files.
- Final logging and monitoring are performed.

Chapter 6

Cryptography Using AES-256-GCM

Introduction

Cryptography plays a crucial role in securing sensitive information during storage and transmission. In this project, Advanced Encryption Standard (AES) with a 256-bit key operating in Galois/Counter Mode (GCM) is used to provide strong encryption and data integrity. AES-256-GCM is widely adopted in government, defence, cloud computing, and secure communication systems due to its high level of security and efficiency.

AES-256 Overview

AES (Advanced Encryption Standard) is a symmetric-key encryption algorithm that uses the same secret key for both encryption and decryption. AES-256 employs a 256-bit key length, making it highly resistant to brute-force attacks and suitable for protecting classified and confidential information.

Galois/Counter Mode (GCM)

GCM is an authenticated encryption mode that combines encryption and authentication in a single operation. In addition to encrypting data, it generates an authentication tag that verifies the integrity and authenticity of the encrypted information.

Working Principle

The AES-256-GCM process consists of the following steps:

1. **Key Generation:** A secure 256-bit encryption key is generated.
2. **Nonce/Initialization Vector (IV) Creation:** A unique IV is generated for each encryption operation.
3. **Encryption:** The plaintext file is encrypted using AES-256.
4. **Authentication Tag Generation:** GCM generates an authentication tag to detect any modification of the encrypted data.
5. **Transmission:** The ciphertext, IV, and authentication tag are transferred to the destination.
6. **Decryption and Verification:** The receiver verifies the authentication tag and decrypts the file only if the verification succeeds.

❖ Advantages of AES-256-GCM

- Strong 256-bit encryption security.
- Provides both confidentiality and integrity.
- Fast and efficient for large files.

- Detects unauthorized modifications.
- Suitable for secure file transfer applications.
- Resistant to common cryptographic attacks.

❖ Application in the Proposed System

In the Secure Offline File Transfer System, AES-256-GCM is used to encrypt files before they are transferred through the intermediate network and Data Diode. The authentication tag ensures that files remain unchanged during transfer, while the encryption protects sensitive information from unauthorized access. This approach guarantees secure, reliable, and tamper-resistant file transmission between networks of different security levels.

Chapter 7

Secure Key Derivation

Introduction

Secure Key Derivation is the process of generating strong cryptographic keys from an initial secret, such as a password, master key, or shared secret. It ensures that encryption keys are difficult to guess and resistant to brute-force attacks. In the proposed Secure Offline File Transfer System, a key derivation mechanism is used to generate secure encryption keys for AES-256-GCM encryption.

Need for Key Derivation

Directly using passwords or user-provided secrets as encryption keys is insecure because they may be weak or predictable. Key Derivation Functions (KDFs) transform these inputs into strong, random-looking cryptographic keys suitable for encryption and authentication.

KeyDerivation Process

The secure key derivation process consists of the following steps:

1. Input Secret Collection

- Obtain a master password or secret key.
- Ensure secure storage and handling of the secret.

2. Salt Generation

- Generate a unique random salt for each operation.
- Prevent precomputed dictionary and rainbow table attacks.

3. Key Derivation Function Execution

- Apply a Key Derivation Function (KDF) such as PBKDF2, HKDF, or Argon2.
- Perform multiple iterations to increase computational difficulty for attackers.

4. Key Generation

- Produce a 256-bit cryptographic key.
- Use the derived key for AES-256-GCM encryption.

5. Secure Storage

- Store only the salt and required parameters.
- Never store plaintext passwords or secret keys.

Common Key Derivation Functions

1. PBKDF2 (Password-Based Key Derivation Function 2)

- Widely used and standardized.
- Uses repeated hashing operations.
- Resistant to brute-force attacks.

2. HKDF (HMAC-based Key Derivation Function)

- Derives multiple cryptographic keys from a single master key.
- Efficient and secure.
- Commonly used in communication protocols.

3. Argon2

- Modern password hashing and key derivation algorithm.
- Provides protection against GPU and hardware-based attacks.
- Recommended for high-security applications.

❖ Application in the Proposed System

In the Secure Offline File Transfer System, the key derivation process generates a strong 256-bit encryption key used by AES-256-GCM. The derived key encrypts files before transfer through the Data Diode. Using secure key derivation ensures that even if a password is weak, the generated encryption key remains highly secure and resistant to attacks.

❖ Benefits

- Enhances cryptographic security.
- Protects against brute-force attacks.
- Prevents password-based vulnerabilities.
- Supports secure file encryption and authentication.
- Improves overall system reliability.

Chapter 8

State Machine Logic

Introduction

A State Machine is a computational model used to control the flow of operations in a system by defining a set of states and transitions between them. In the proposed Secure Offline File Transfer System, the state machine logic manages the complete lifecycle of a file, from upload to final delivery in the secure LAN. This approach ensures that files progress through predefined security checkpoints and prevents unauthorized transfers.

States of the System

1. Idle State

The system waits for a file transfer request. No processing occurs until a valid file is submitted.

Action:

- Monitor for upload requests.
- Initialize system resources.

2. File Upload State

A user uploads a file from the Internet zone.

Action:

- Verify user authentication.
- Record file metadata.
- Store file in the staging area.

3. Security Scan State

The uploaded file undergoes malware and policy checks.

Action:

- Antivirus scanning.
- File type validation.
- Threat detection.

Transition:

- If the file is safe, proceed to Validation State.
- If malicious, move to Rejected State.

4. Validation State

The system verifies organizational security policies.

Action:

- Check file permissions.
- Verify file source.
- Confirm transfer authorization.

Transition:

- Valid files proceed to Encryption State.
- Invalid files move to Rejected State.

5. Encryption State

The file is encrypted using AES-256-GCM.

Action:

- Generate encryption key.
- Encrypt file contents.
- Generate authentication tag.

Transition:

- Successfully encrypted files proceed to Transfer State.

6. Transfer State

The encrypted file is transferred through the Data Diode.

Action:

- Enable one-way transfer.
- Log transfer activity.
- Prevent reverse communication.

7.Verification State

The destination system verifies the transferred file.

Action:

- Validate authentication tag.
- Verify SHA-256 hash.
- Confirm file integrity.

Transition:

- Verified files proceed to Delivery State.
- Failed verification moves to Rejected State.

8.Delivery State

The file becomes available within the secure LAN.

Action:

- Store file securely.
- Notify authorized users.
- Update audit logs.

9.Rejected State

Files failing security or integrity checks are blocked.

Action:

- Quarantine file.
- Generate security alert.
- Record failure reason.

10.Completed State

The file transfer process ends successfully.

Action:

- Archive logs.
- Release system resources.

❖ Advantages of State Machine Logic

- Provides structured workflow management.
- Prevents unauthorized state transitions.
- Improves system reliability and security.
- Simplifies debugging and monitoring.
- Ensures every file passes mandatory security checks.

Chapter 9

Module Structure

Introduction

The Secure Offline File Transfer System is divided into multiple functional modules. Each module performs a specific task in the file transfer process, ensuring security, integrity, authentication, and controlled data movement from the Internet zone to the Secure LAN. The modular design improves maintainability, scalability, and reliability.

Module 1: User Authentication and Access Control:-

Functions

- User login and authentication.
- Role-Based Access Control (RBAC).
- Session management.
- User activity tracking.

Purpose

To ensure that only authorized users can upload and access files within the system.

Module 2: File Upload and Staging Module:-

Functions

- File selection and upload.
- Metadata collection.
- Temporary file storage.
- Upload validation.

Purpose

To securely receive files from the Internet zone and prepare them for security analysis.

Module 3: Security Scanning Module:-

Functions

- Malware detection.
- Antivirus scanning.
- File type verification.
- Threat analysis.

Purpose

To identify and block malicious or unauthorized files before transfer.

Module 4: Encryption and Key Management Module:-

Functions

- Secure key derivation.
- AES-256-GCM encryption.
- Authentication tag generation.
- Encryption key management.

Purpose

To protect file confidentiality and integrity during transfer.

Module 5: Data Diode Transfer Module:-

Functions

- One-way file transmission.
- Secure transfer control.
- Transfer monitoring.
- Communication isolation.

Purpose

To enforce unidirectional data flow from the Internet network to the secure network.

Module 6: Integrity Verification Module:-

Functions

- SHA-256 hash generation.
- Hash comparison.
- Authentication tag verification.
- File validation.

Purpose

To ensure that files have not been modified during transfer.

Module 7: Secure Storage and Delivery Module:-

Functions

- Secure file storage.
- Controlled file access.
- User notification.
- Repository management.

Purpose

To safely deliver verified files to authorized users within the LAN.

Module 8: Audit Logging and Monitoring Module:-

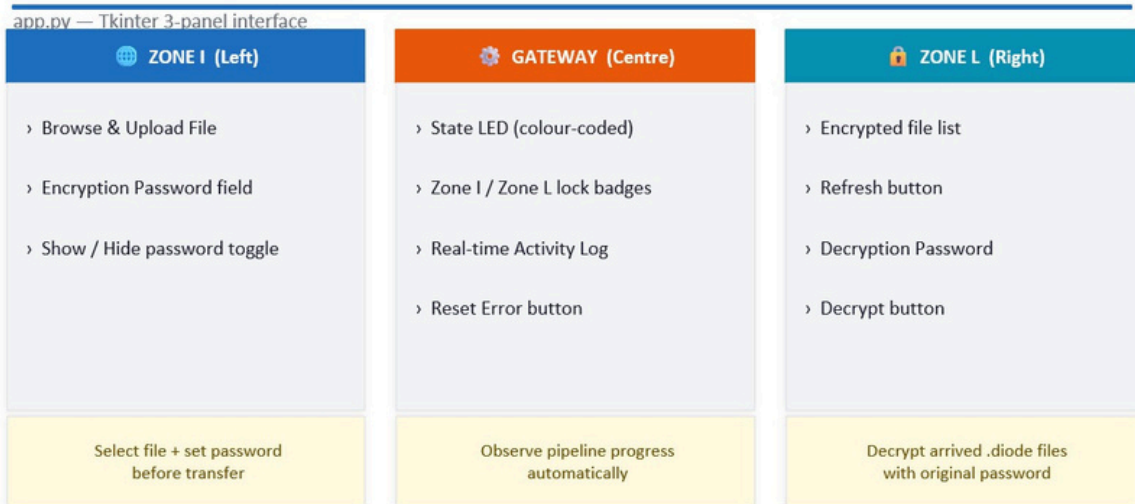
Functions

- Activity logging.
- Event monitoring.
- Security alerts.
- Transfer history management.

Chapter 10

10.1 GUI Walkthrough

GUI Walkthrough



The figure illustrates the three-panel Graphical User Interface (GUI) of the Offline Secure File Transfer System. The interface is divided into three logical sections representing Zone I (Source Network), the Gateway, and Zone L (Destination Network). This design provides a clear visualization of the secure file transfer pipeline and enables users to monitor file movement across isolated network zones.

1. Zone I (Left Panel – Source Network)

The left panel is used by the sender to prepare files for transfer. It contains:

1. Browse & Upload File option for selecting the file.
2. Encryption Password Field for entering a secure password.
3. Show/Hide Password Toggle for password visibility control.
4. The user first selects a file and provides an encryption password before initiating the transfer process.

2. Gateway (Center Panel)

The center panel acts as the secure transfer controller and monitoring unit. It includes:

1. State LED Indicator displaying the current system status using color codes.

2. Zone I / Zone L Lock Badges indicating network isolation and security status.
3. Real-Time Activity Log showing ongoing transfer operations.
4. Reset Error Button for clearing system errors and restoring normal operation.

This panel allows administrators to observe the secure transfer pipeline and monitor system events in real time.

3. Zone L (Right Panel – Destination Network)

The right panel is used by the receiving side to access transferred files. It provides: Encrypted File List displaying files received from the gateway.

Refresh Button to update the available file list.

Decryption Password Field for entering the correct password.

Decrypt Button to recover the original file.

10.2 GUI TRANSFER WALKTHROUGH:-



The figure presents the step-by-step workflow of the secure file transfer process in the Offline Secure File Transfer System. It demonstrates how a file is securely transferred from Zone I (Source Network) to Zone L (Destination Network) through the Gateway while maintaining confidentiality and integrity.

Step 1: Enter Encryption Password

The user begins by entering a strong password in the Zone I password field. This password is later used to encrypt the selected file before transfer.

Step 2: Browse and Upload File

The user clicks the Browse & Upload File button and selects the file to be transferred. Once selected, the file is placed in the source zone for processing.

Step 3: Automatic File Detection

A background monitoring service (watchdog) continuously observes the upload directory. When a new file is detected, the secure transfer pipeline starts automatically without requiring further user intervention.

Step 4: Gateway Processing

The Gateway receives the uploaded file and updates its status through different states:

IDLE – Waiting for a file.

RECEIVING – Accepting the encrypted file from Zone I.

SENDING – Forwarding the encrypted file to Zone L.

IDLE – Returning to standby mode after successful transfer.

This stage provides real-time monitoring of the transfer operation.

Step 5: File Arrival in Zone L

After successful transmission, the encrypted file appears in the Zone L encrypted file list, indicating that the transfer has been completed successfully.

Step 6: File Decryption

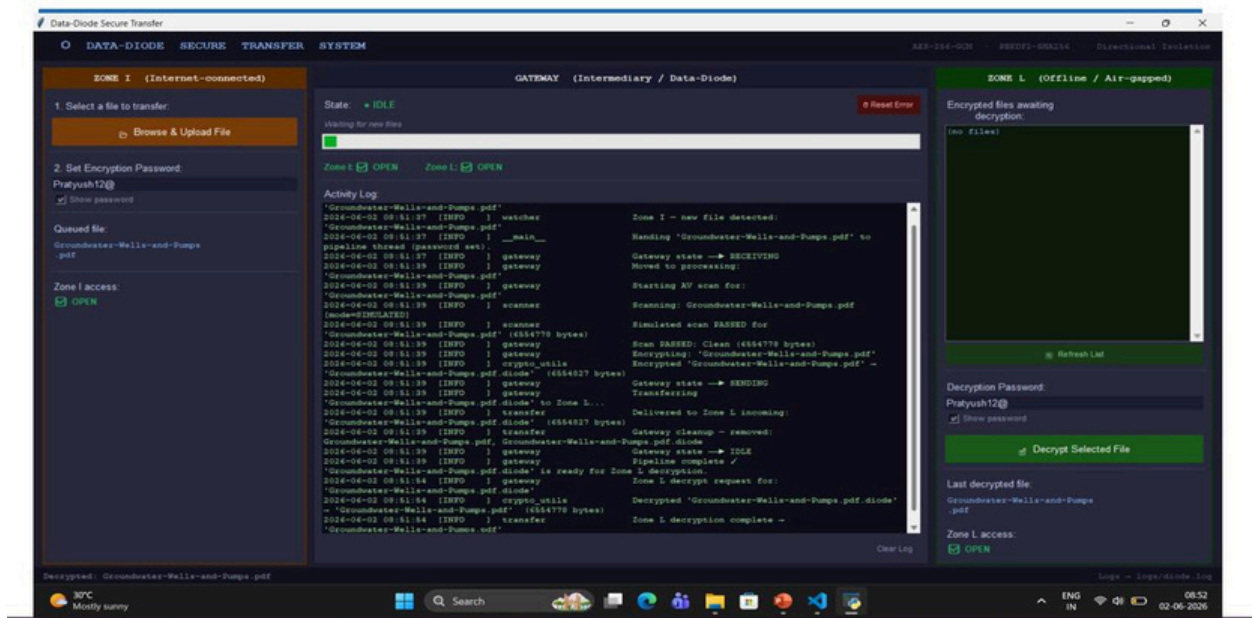
The receiver enters the same password used during encryption, selects the transferred file, and clicks the Decrypt button. The system then performs AES-256-GCM decryption to recover the original file.

Step 7: Save Decrypted File

The decrypted file is stored in the designated destination folder (e.g., /zone_1/decrypted/), making it available for authorized use within the destination network.

10.3 GUI MODE

GUI Mode



Chapter 11

CLI MODE:-

The CLI (Command Line Interface) Mode provides a text-based interface for secure file transfer operations. It is designed for administrators and advanced users who prefer command-driven execution instead of graphical controls.

Features

1. Lightweight and resource-efficient operation.
2. Supports automated scripting and batch processing.
3. Enables secure file transfer through terminal commands.
4. Provides detailed status messages and logs.
5. Suitable for server environments where a GUI is unavailable.

Workflow

1. User launches the application in CLI mode.
2. System prompts for source file and destination details.
3. Security checks are performed on the selected file.
4. File encryption is applied using AES-256-GCM.
5. The encrypted file is transferred through the designated channel.
6. Transfer status and verification results are displayed in the terminal.
7. Logs are generated for auditing and troubleshooting purposes.

Advantages

1. Faster execution with minimal system overhead.
2. Easy integration with shell scripts and automation tools.
3. Improved control over transfer parameters.
4. Useful for remote administration and secure environments.

```

(.venv) PS C:\Users\praty\Downloads\data_diode_transfer\data_diode_transfer> python cli_runner.py
Gateway state : IDLE
Zone I queue : DS000027 (1).BMP.diode
> decrypt
Encrypted files in Zone I:
[1] DS000027 (1).BMP.diode
Select number (or 0 to cancel): 1
Decryption password for 'DS000027 (1).BMP.diode': Pratyush12@
2026-06-02 08:47:18 [INFO ] gateway      Zone I decrypt request for: 'DS000027 (1).BMP.diode'
2026-06-02 08:47:18 [INFO ] crypto_utils  Decrypted 'DS000027 (1).BMP.diode' + 'DS000027 (1).BMP' (768066 bytes)
2026-06-02 08:47:18 [INFO ] transfer      Zone I decryption complete + 'DS000027 (1).BMP'

✅ Decrypted successfully + C:\Users\praty\Downloads\data_diode_transfer\data_diode_transfer\zones\zone_1\decrypted\DS000027 (1).BMP
> quit
2026-06-02 08:47:34 [INFO ] watcher      Zone I watcher stopped.
Watcher stopped. Goodbye.

```

***CLI (Command Line Interface) Mode**

Chapter 12

Threading Model

Introduction

The Threading Model defines how multiple tasks are executed concurrently within the Secure Offline File Transfer System. By using multiple threads, the system can perform file uploads, security scanning, encryption, transfer, verification, and logging simultaneously, improving performance and responsiveness.

Need for Multithreading

In a secure file transfer environment, several operations occur independently and may take significant processing time. A multithreaded architecture ensures that one operation does not block others and allows efficient utilization of system resources.

Benefits

- Improved system performance.
- Faster file processing.
- Better resource utilization.
- Enhanced user experience.
- Support for multiple file transfers.

Thread Structure

1. User Interface Thread

Responsible for handling user interactions.

Functions

- User login.
- File selection.
- Displaying transfer status.
- Showing notifications and alerts.

2. File Upload Thread

Handles file upload operations.

Functions

- Receive files from users.
- Store files in the staging area.
- Manage upload queues.

3.Security Scanning Thread

Performs malware and policy checks.

Functions

- Antivirus scanning.
- File type validation.
- Threat detection.

4.Encryption Thread

Encrypts files using AES-256-GCM.

Functions

- Generate encryption keys.
- Encrypt file contents.
- Create authentication tags.

5.Transfer Thread

Manages Data Diode-based file transfer.

Functions

- Transfer encrypted files.
- Monitor transfer progress.
- Handle transmission errors.

6.Verification Thread

Validates transferred files.

Functions

- SHA-256 hash verification.
- Authentication tag validation.
- Integrity checking.

7. Audit Logging Thread

Records system activities.

Functions

- Log user actions.
- Store transfer records.
- Generate audit reports.

Thread Synchronization

To prevent conflicts between multiple threads, synchronization mechanisms are used:

Techniques

- Mutex Locks
- Semaphores
- Thread-safe Queues
- Event Signals

Purpose

- Prevent data corruption.
- Ensure correct execution order.
- Maintain file consistency.

Advantages of the Proposed Threading Model

- Supports concurrent processing.
- Reduces overall transfer time.
- Improves system scalability.
- Maintains secure and reliable operations.
- Enables efficient handling of multiple files.

Chapter 13

ERROR HANDLING:-

The Error Handling Module ensures that all failure conditions during the secure file transfer process are detected, managed, and reported appropriately. It prevents system crashes, maintains workflow integrity, and provides clear recovery actions for the operator.

Error Conditions and Handling

Error Condition	Exception/Detection	System State	Operator Action
Wrong decryption Password	ValueError	Unchanged	Re-enter the correct password
Corrupted data-diode File	ValueError	Unchanged	Re-transfer the file from Zone 1
Threat detected by scanner	Handled internally	ERROR	Inspect quarantined
Empty file uploaded	Handled internally	ERROR	Upload a valid non-empty file
Duplicate file in processing	FileExistsError	ERROR	Remove duplicate file

Key Features

1. Detects cryptographic validation failures.
2. Handles malware detection events safely.
3. Prevents duplicate or invalid file submissions.
4. Maintains system stability during concurrent operations.
5. Provides clear recovery instructions to operators.
6. Ensures secure state transitions without data loss.

Chapter 14

Code :-

Threading model

Main thread : Tkinter event loop (all UI mutations happen here)
Watcher thread: watchdog Observer (daemon)
Pipeline thread: one daemon thread per file (gateway.process_file)
All cross-thread UI calls go through root.after(0, fn) – Tkinter is not thread-safe; this keeps every widget update on the main thread.

"""

```
import threading
import shutil
import tkinter as tk
from tkinter import ttk, scrolledtext, filedialog, messagebox
from pathlib import Path

from config import (
    APP_TITLE, APP_GEOMETRY, FONT_MONO, FONT_LABEL, FONT_TITLE,
    COLOUR_BG, COLOUR_PANEL, COLOUR_TEXT, COLOUR_ACCENT,
    COLOUR_IDLE, COLOUR_RECEIVING, COLOUR_SENDING, COLOUR_ERROR,
    GatewayState, ZONE_I_INPUT, ZONE_L_INCOMING, ALL_DIRS,
)

from logger_setup import get_logger, set_ui_callback
from gateway import Gateway
from watcher import ZoneIWatcher

log = get_logger(__name__)

# GatewayState → indicator colour
STATE_COLOURS = {
    GatewayState.IDLE: COLOUR_IDLE,
    GatewayState.RECEIVING: COLOUR_RECEIVING,
    GatewayState.SENDING: COLOUR_SENDING,
    GatewayState.ERROR: COLOUR_ERROR,
}

STATE_DESCRIPTIONS = {
    GatewayState.IDLE: "Waiting for new files",
    GatewayState.RECEIVING: "Processing Zone I file (Zone L locked)",
    GatewayState.SENDING: "Transferring to Zone L (Zone I locked)",
    GatewayState.ERROR: "Error – check log and press Reset",
}
```

```

# -----
# Main application class
# -----

class DataDiodeApp:

    def __init__(self, root: tk.Tk):
        self.root = root
        self.gateway = Gateway()
        self._ensure_dirs()
        self._build_ui()
        self._wire_callbacks()
        self._start_watcher()
        log.info("Data-Diode Transfer application started.")

# --- Setup ---

def _ensure_dirs(self):
    """Create zone directories if they don't exist (idempotent)."""
    for d in ALL_DIRS:
        Path(d).mkdir(parents=True, exist_ok=True)

# -----
# UI construction
# -----

def _build_ui(self):
    self.root.title(APP_TITLE)
    self.root.geometry(APP_GEOMETRY)
    self.root.configure(bg=COLOUR_BG)
    self.root.minsize(820, 560)

    self._build_title_bar()
    self._build_main_panels()
    self._build_status_bar()

# --- Title bar ---

def _build_title_bar(self):
    bar = tk.Frame(self.root, bg=□ "#12121e", height=42)
    bar.pack(fill=tk.X)
    bar.pack_propagate(False)

    tk.Label(
        bar,
        text=" O DATA-DIODE SECURE TRANSFER SYSTEM",
        font=("Courier", 12, "bold"),
        fg=COLOUR_ACCENT, bg=□ "#12121e",
    ).pack(side=tk.LEFT, padx=12, pady=8)

    tk.Label(
        bar,
        text="AES-256-GCM · PBKDF2-SHA256 · Directional Isolation",
        font=("Courier", 8),
        fg=□ "#585878", bg=□ "#12121e",
    ).pack(side=tk.RIGHT, padx=12)

```

```

# --- Three-panel body ---

def _build_main_panels(self):
    body = tk.Frame(self.root, bg=COLOUR_BG)
    body.pack(fill=tk.BOTH, expand=True, padx=8, pady=(6, 0))
    body.columnconfigure(0, weight=1, minsize=220)
    body.columnconfigure(1, weight=2, minsize=300)
    body.columnconfigure(2, weight=1, minsize=220)
    body.rowconfigure(0, weight=1)

    self._build_zone_i_panel(body)
    self._build_gateway_panel(body)
    self._build_zone_l_panel(body)

def _make_panel(self, parent, col: int, header: str, hdr_bg: str) -> tk.Frame:
    """Create a standard panel: coloured header strip + dark content area."""
    outer = tk.Frame(parent, bg=hdr_bg)
    outer.grid(row=0, column=col, sticky="nsew", padx=4, pady=4)
    outer.columnconfigure(0, weight=1)
    outer.rowconfigure(1, weight=1)

    tk.Label(
        outer, text=header,
        font=("Courier", 10, "bold"),
        fg="white", bg=hdr_bg, pady=5,
    ).grid(row=0, column=0, sticky="ew")

    inner = tk.Frame(outer, bg=COLOUR_PANEL)
    inner.grid(row=1, column=0, sticky="nsew", padx=2, pady=(0, 2))
    inner.columnconfigure(0, weight=1)
    return inner

```

```

# — Zone I panel —————

def _build_zone_i_panel(self, parent):
    f = self._make_panel(parent, col=0,
                          header=" ZONE I (Internet-connected)",
                          hdr_bg=COLOUR_PANEL)

    self._lbl(f, "1. Select a file to transfer:").pack(padx=10, pady=(14, 2), anchor=tk.W)

    tk.Button(
        f, text="📁 Browse & Upload File",
        command=self._on_upload_file,
        bg=COLOUR_PANEL, fg="white", font=FONT_LABEL,
        relief=tk.FLAT, padx=8, pady=7, cursor="hand2",
        activebackground=COLOUR_PANEL, activeforeground="white",
    ).pack(padx=10, pady=4, fill=tk.X)

    self._sep(f)

    self._lbl(f, "2. Set Encryption Password:").pack(padx=10, pady=(8, 2), anchor=tk.W)
    self.enc_pwd_var = tk.StringVar()
    enc_entry = self._pwd_entry(f, self.enc_pwd_var)
    enc_entry.pack(padx=10, fill=tk.X)
    self._show_toggle(f, enc_entry).pack(padx=10, anchor=tk.W, pady=(0, 2))

    self._sep(f)
    self._lbl(f, "Queued file:").pack(padx=10, pady=(8, 2), anchor=tk.W)
    self.queued_file_var = tk.StringVar(value="-- (none)")
    tk.Label(
        f, textvariable=self.queued_file_var,
        font=("Courier", 8), fg=COLOUR_ACCENT, bg=COLOUR_PANEL,
        wraplength=195, justify=tk.LEFT,
    ).pack(padx=10, anchor=tk.W, pady=(0, 10))

    self._sep(f)
    self._lbl(f, "Zone I access:").pack(padx=10, pady=(6, 0), anchor=tk.W)
    self._zone_i_badge = tk.Label(
        f, text="🟢 OPEN", font=("Helvetica", 9, "bold"),
        fg=COLOUR_PANEL, bg=COLOUR_PANEL,
    )
    self._zone_i_badge.pack(padx=10, anchor=tk.W, pady=(0, 10))

```



```

# — Gateway panel —

def _build_gateway_panel(self, parent):
    f = self._make_panel(parent, col=1,
                          header=" GATEWAY (Intermediary / Data-Diode)",
                          hdr_bg=COLOUR_PANEL)

    # State row
    state_row = tk.Frame(f, bg=COLOUR_PANEL)
    state_row.pack(fill=tk.X, padx=10, pady=(10, 4))

    self._lbl(state_row, "State:").pack(side=tk.LEFT)

    self._state_led = tk.Label(
        state_row, text=" • IDLE ",
        font=("Helvetica", 10, "bold"),
        fg=COLOUR_IDLE, bg=COLOUR_PANEL,
    )
    self._state_led.pack(side=tk.LEFT, padx=6)

    tk.Button(
        state_row, text="⏮ Reset Error",
        command=self._on_reset_error,
        bg=COLOUR_PANEL, fg=COLOUR_PANEL,
        font=("Helvetica", 8), relief=tk.FLAT,
        padx=5, pady=2, cursor="hand2",
        activebackground=COLOUR_PANEL,
    ).pack(side=tk.RIGHT)

    self._state_desc = tk.Label(
        f, text="Waiting for new files",
        font=("Helvetica", 8, "italic"),
        fg=COLOUR_PANEL, bg=COLOUR_PANEL,
    )
    self._state_desc.pack(padx=10, anchor=tk.W)

    # Pipeline progress bar
    self._progress = ttk.Progressbar(f, mode="indeterminate", length=200)
    self._progress.pack(padx=10, pady=(4, 2), fill=tk.X)

    self._sep(f)

    # Zone lock badges
    badge_row = tk.Frame(f, bg=COLOUR_PANEL)
    badge_row.pack(fill=tk.X, padx=10, pady=4)
    self._zone_i_gw_badge = tk.Label(
        badge_row, text="Zone I: ☒ OPEN",
        font=("Helvetica", 9, "bold"), fg=COLOUR_PANEL, bg=COLOUR_PANEL,
    )
    self._zone_i_gw_badge.pack(side=tk.LEFT, padx=(0, 30))
    self._zone_i_badge = tk.Label(
        badge_row, text="Zone I: ☒ OPEN",
        font=("Helvetica", 9, "bold"), fg=COLOUR_PANEL, bg=COLOUR_PANEL,
    )
    self._zone_i_badge.pack(side=tk.LEFT)

    self._sep(f)

    # Log pane
    self._lbl(f, "Activity Log:").pack(padx=10, pady=(4, 2), anchor=tk.W)

    log_frame = tk.Frame(f, bg=COLOUR_PANEL)
    log_frame.pack(padx=10, pady=(0, 10), fill=tk.BOTH, expand=True)
    log_frame.columnconfigure(0, weight=1)
    log_frame.rowconfigure(0, weight=1)

    self.log_text = scrolledtext.ScrolledText(
        log_frame,
        font=("Courier", 8),
        bg=COLOUR_PANEL, fg=COLOUR_PANEL,
        insertbackground=COLOUR_PANEL,
        relief=tk.FLAT, wrap=tk.WORD,
        state=tk.DISABLED,
    )
    self.log_text.pack(fill=tk.BOTH, expand=True)

```



```

# --- Zone L panel ---

def _build_zone_l_panel(self, parent):
    f = self._make_panel(parent, col=2,
                          header=" ZONE L (Offline / Air-gapped)",
                          hdr_bg="#0a3a0a")

    self._lbl(f, "Encrypted files awaiting\ndecryption:").pack(
        padx=10, pady=(12, 2), anchor=tk.W
    )

    list_frame = tk.Frame(f, bg=COLOUR_PANEL)
    list_frame.pack(padx=10, fill=tk.BOTH, expand=True)

    sb = tk.Scrollbar(list_frame)
    sb.pack(side=tk.RIGHT, fill=tk.Y)

    self.file_listbox = tk.Listbox(
        list_frame,
        yscrollcommand=sb.set,
        bg="#0e1a0e", fg=COLOUR_ACCENT,
        selectbackground="#2a5a2a",
        selectforeground="white",
        font=("Courier", 8),
        relief=tk.FLAT,
        selectmode=tk.SINGLE,
        activestyle="none",
    )
    self.file_listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
    sb.config(command=self.file_listbox.yview)

    tk.Button(
        f, text="🔄 Refresh List",
        command=self._refresh_zone_l_list,
        bg="#1a3a1a", fg="#88cc88",
        font=("Helvetica", 8), relief=tk.FLAT, padx=6, pady=3,
        cursor="hand2",
    ).pack(padx=10, pady=4, fill=tk.X)

    self._sep(f)

    self._lbl(f, "Decryption Password:").pack(padx=10, pady=(8, 2), anchor=tk.W)
    self.dec_pwd_var = tk.StringVar()
    dec_entry = self._pwd_entry(f, self.dec_pwd_var)
    dec_entry.pack(padx=10, fill=tk.X)
    self._show_toggle(f, dec_entry).pack(padx=10, anchor=tk.W, pady=(0, 2))

    tk.Button(
        f, text="🔓 Decrypt Selected File",
        command=self._on_decrypt,
        bg="#1a5a1a", fg="white",
        font=FONT_LABEL, relief=tk.FLAT,
        padx=8, pady=7, cursor="hand2",
        activebackground="#2a7a2a",
    ).pack(padx=10, pady=(10, 4), fill=tk.X)

    self._sep(f)
    self._lbl(f, "Last decrypted file:").pack(padx=10, pady=(6, 2), anchor=tk.W)
    self.dec_result_var = tk.StringVar(value="-- (none)")
    tk.Label(
        f, textvariable=self.dec_result_var,
        font=("Courier", 8), fg=COLOUR_ACCENT,
        bg=COLOUR_PANEL, wraplength=195, justify=tk.LEFT,
    ).pack(padx=10, anchor=tk.W, pady=(0, 10))

    self._lbl(f, "Zone L access:").pack(padx=10, pady=(4, 0), anchor=tk.W)
    self._zone_l_access_badge = tk.Label(
        f, text="🟢 OPEN",
        font=("Helvetica", 9, "bold"), fg="#27ae60", bg=COLOUR_PANEL,
    )
    self._zone_l_access_badge.pack(padx=10, anchor=tk.W, pady=(0, 10))

```

Chapter 15

Conclusion and Future Scope

15.1 Conclusion

The Secure Offline File Transfer System Between Air-Gapped Networks provides a reliable and secure mechanism for transferring files from an external network to a highly protected internal network without establishing a direct network connection. The system employs a three-zone architecture, AES-256-GCM encryption, malware scanning, integrity verification, and controlled transfer workflows to ensure confidentiality, integrity, and authenticity of transferred files.

By implementing strict validation and security checks at each stage of the transfer pipeline, the system minimizes the risk of malware infiltration, unauthorized access, and data tampering. The inclusion of GUI and CLI interfaces, state-machine-based control logic, and comprehensive error-handling mechanisms enhances usability, maintainability, and operational reliability. The proposed solution demonstrates how sensitive organizations such as defense, government, and critical infrastructure sectors can securely exchange information across isolated networks while preserving the security benefits of air-gapped environments.

15.2 Future scoop

❖ Security & Cryptography

- Hardware Data Diode Integration: Replace the software-based simulation with a physical one-way data diode device.
- Digital Signature Support: Implement digital signatures for non-repudiation and authenticity.
- Role-Based Access Control (RBAC): Introduce multi-level user roles and permissions.
- Hardware Security Module (HSM) Integration: Offload cryptographic key management to a dedicated HSM.

❖ Threat Detection & Intelligence

- Artificial Intelligence-Based Threat Detection: Utilize machine learning models to identify anomalous files.
- Advanced Malware Analysis: Integrate sandboxing and behavioral analysis to detect sophisticated threats.
- Federated Threat Intelligence (Offline): Enable sharing of sanitized threat signatures across air-gapped installations.

❖ **Operations & Performance**

- Multi-File and Large-Scale Transfer Support: Optimize the system for batch processing and high-volume data.
- Cross-Platform Deployment: Extend support to Linux, Windows, and embedded systems.
- Automated Threat Intelligence Updates: Enable secure offline updates of malware signatures and policies.

❖ **Monitoring & Governance**

- Comprehensive Audit and Logging: Develop centralized audit trails and reporting dashboards.
- Blockchain-Based Audit Records: Store transfer logs in a tamper-resistant blockchain ledger.
- Mobile Monitoring Application: Develop an application for administrators to monitor system health.
- Cloud-Assisted Offline Management: Provide secure synchronization of logs through controlled update packages.

Chapter 16

REFERENCES:-

1. Alves, T., and Sherwood, R., "The Design and Evaluation of a Multi-Level Security Architecture for Critical Infrastructure Control Systems," Proceedings of the 44th Hawaii International Conference on System Sciences (HICSS), 2011.
2. Anderson, R., "Security Engineering: A Guide to Building Dependable Distributed Systems," 2nd ed., Wiley, 2008.
3. Dworkin, M. J., "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007.
4. Turan, M. S., Barker, E., Burr, W., and Chen, L., "Recommendation for Password-Based Key Derivation: Part 1: Storage Applications," NIST Special Publication 800-132, National Institute of Standards and Technology, 2010.
5. Bellare, M., Canetti, R., and Krawczyk, H., "Keying Hash Functions for Message Authentication," Advances in Cryptology—CRYPTO '96, Lecture Notes in Computer Science, Springer, 1997.
6. McGrew, D. A., and Viega, J., "The Security and Performance of the Galois/Counter Mode (GCM) of Operation," Proceedings of the International Workshop on Fast Software Encryption, 2004.
7. Kaliski, B., and Staddon, J., "PKCS #5: Password-Based Cryptography Specification Version 2.0," RFC 2898, Internet Engineering Task Force (IETF), 2000.
8. OWASP Foundation, "Password Storage Cheat Sheet," Open Web Application Security Project (OWASP), 2023.
9. Zuboff, S., and Maxmin, J., "The Support Economy: Why Corporations Are Failing Individuals and the Next Episode of Capitalism," Harvard Business Press, 2002.